

通过元学习进行深度在线学习：基于模型的强化学习的持续适应

Anusha Nagabandi, Chelsea Finn, Sergey Levine {
nagaban2, cbfinn, sergey.levine}@berkeley.edu 加州大学伯克利分校

抽象的

人类和动物可以学习复杂的预测模型，使他们能够准确可靠地推理现实世界的现象，并且在面对意外变化时可以极快地适应这些模型。深度神经网络模型使我们能够表示非常复杂的函数，但缺乏快速在线适应的能力。本文的目标是开发一种使用深度神经网络模型从输入数据流中持续在线学习的方法。我们制定了一个在线学习程序，使用随机梯度下降来更新模型参数，并在开发和维护混合模型来处理非平稳任务分布之前，使用中餐馆流程的期望最大化算法。这允许根据需要调整所有模型，为任务更改实例化新模型，并在再次遇到以前看到的任务时调用旧模型。此外，我们观察到元学习可用于对模型进行元训练，使得这种使用 SGD 的直接在线适应是有效的，而对于大型函数逼近器则不然。在这项工作中，我们将在线学习元学习（MOLe）方法应用于基于模型的强化学习，其中调整预测模型对于控制至关重要；我们证明 MOLe 优于其他现有方法，并且能够有效地连续适应非平稳任务分布，例如变化的地形、电机故障和意外干扰。视频可访问：<https://sites.google.com/berkeley.edu/onlineviameta>

1 简介

人类和动物学习的特点不仅在于获得复杂技能的能力，而且还在于必须新的或不断变化的条件下执行这些技能时快速适应的能力。例如，动物可以快速适应在不同表面上行走和跑步（Herman, 2017），而人类可以在出现意外扰动轻松调节伸手运动过程中的力量（Flanagan & Wing, 1993）。此外，这些经历会被记住，并且当将来发生类似的干扰时可以回忆起来更快地适应（Doyon & Benali, 2005）。由于在如此短的时间尺度上学习全新的模型是不切实际的，因此我们可以设计显式训练模型以快速适应少量数据的算法。这种在线适应对于在现实世界中运行的智能系统至关重要，在现实世界中，不断变化的因素和意外的扰动是常态。在本文中，我们提出了一种快速、连续的在线学习算法，该算法利用深度神经网络模型来构建和维护任务分布，从而允许泛化和任务专业化的自然发展。

我们的工作示例是基于模型的强化学习设置中的持续适应，尽管我们的方法通常可以解决任何具有流数据的在线学习场景。我们假设每个“试验”由多个任务组成，并且任务之间的描述没有明确地提供给学习者 - 相反，该方法必须自适应地决定“任务”甚至代表什么，何时实例化新任务，以及何时继续更新旧任务。例如，在变化的地形上运行的机器人可能需要处理上坡和下坡，并且可能

选择维护专门针对每个坡度的单独模型，并根据当前推断的表面依次适应每个坡度。

我们简单地通过对模型参数使用在线随机梯度下降 (SGD) 来执行自适应，同时针对不同任务的模型参数维护混合模型。该混合物通过中餐馆流程进行更新 (Stimberg et al., 2012)，这使得新任务能够在试验过程中根据需要实例化。尽管在线学习可能是 SGD 最古老的应用之一 (Bottou, 1998)，但使用这种方法在线训练现代参数模型 (例如深度神经网络) 非常困难。它们通常需要中等大小的小批量和多个时期才能得出合理的解决方案，这在在线流设置中接收数据时不适合。我们的主要观察之一是，元学习可用于学习参数的事先初始化，从而使这种直接在线适应变得可行，只需少量的梯度步骤。我们使用的元训练过程基于与模型无关的元学习 (MAML) (Finn et al., 2017)，其中学习模型的先前权重初始化，以便在少量梯度步骤后优化元训练任务分布中任何任务的改进。

MAML 的元学习之前已扩展到基于模型的 RL (Nagabandi 等人, 2018)，但仅适用于 k -shot 适应设置：元学习的先验模型适应 k 最近的时间步，但适应不会及时推进 (即，适应始终从先验本身执行)。这种严格的批处理模式设置在在线学习设置中受到限制，并且不足以完成训练分布之外的任务。一种更自然的表述是模型接收连续的数据流，并且必须在线适应潜在的非平稳任务分布。这需要快速适应和回忆先前任务的能力，以及根据需要在两者之间进行插值的有效适应策略。

本文的主要贡献是一种在线学习元学习 (MOLe) 算法，该算法使用期望最大化，结合任务分配之前的中餐馆流程来学习神经网络模型的混合物，每个模型都使用在线 SGD 进行更新。与之前的多任务和元学习方法相比，我们的方法在线分配软任务概率允许任务专门化自然出现，而不需要提前指定任务描述。我们在基于模型的 RL 背景下对一系列具有挑战性的模拟机器人任务 (包括干扰、环境变化和模拟电机故障) 评估 MOLe。我们的模拟实验展示了半猎豹智能体和六足爬行机器人在在线环境中执行连续模型自适应。我们的结果显示了新任务的在线实例化、适应分布外任务的能力以及识别和恢复到先前任务的能力。此外，我们证明 MOLe 优于最先进的现有方法，该方法执行基于 k -shot 模型的元强化学习，以及自然基线，例如用于适应和在线学习的连续梯度更新，无需元训练。

2 相关工作

在线学习是机器学习最古老的子领域之一 (Bottou, 1998; Jafari 等人, 2001)。先前的算法使用在线梯度更新 (Duchi 等人, 2011) 和概率过滤公式 (Murphy, 2002; Hoffman 等人, 2010; Broderick 等人, 2013)。原则上，常用的基于梯度的学习方法，例如 SGD，可以很容易地用作在线学习算法 (Bottou, 1998)。在实践中，它们在深度神经网络函数逼近器方面的性能是有限的 (Sahoo et al., 2017)：此类高维模型必须使用批处理模式方法、小批量和多次传递数据进行训练。我们的目标是通过使用与模型无关的元学习 (MAML) 显式预训练一个能够快速适应的模型来解除这一限制，然后我们通过中餐馆流程的期望最大化算法 (Blei et al., 2003) 将其用于连续在线适应，然后在非平稳任务分布中动态分配新任务。

在线学习与持续学习或终身学习相关 (Thrun, 1998)，其中代理随着时间的推移面临非平稳的任务分配。然而，与专注于避免负迁移 (即灾难性遗忘) 的研究不同 (Kirkpatrick et al., 2017; Rebuffi et al., 2017; Zenke et al., 2017; Lopez-Paz et al., 2017; Nguyen et al., 2017)，在线学习侧重于在非平稳性存在下快速学习和适应的能力。虽然一些持续学习有效

考虑前向传输的问题，例如鲁苏等人。（2016）；阿尔琼迪等人。（2017）；王等人。（2017），这些工作和其他持续学习的工作通常集中于小任务集，在这些任务中，快速、在线学习实际上是不可能的，因为根本没有足够的任务来恢复结构，从而在新任务或环境中实现快速、少量的学习。

我们的方法建立在元学习或学习中学习的技术之上（Thrun & Pratt, 1998; Schmidhuber, 1987; Bengio et al., 1992; Naik & Mammone, 1992）。然而，最近的元学习工作考虑的是一次学习一个任务的设置，通常是从一批数据中学习（Santoro et al., 2016; Ravi & Larochelle, 2017; Munkhdalai & Yu, 2017; Wang et al., 2016; Duan et al., 2016）。在我们的工作中，我们专门解决非平稳任务分布，并且不假设任务边界是已知的。之前的工作（Jerfel 等人，2018）也考虑了非平稳任务分配；而杰菲尔等人。（2018）使用元梯度来估计特定于任务参数的混合物参数，我们专注于在运行时优化期间特定于任务的混合分量的快速适应和累积。其他元学习工作考虑了任务内的非平稳性（Al-Shedivat et al., 2017）和元测试时涉及多个任务的事件（Ritter et al., 2018），但他们没有考虑未知任务分离的持续在线适应。之前的工作还研究了基于模型的强化学习的元学习（Nagabandi 等人，2018）。此先前方法在每个时间步骤更新模型，但每次更新都是批量模式 k -shot 更新，精确使用 k 先前转换并在每个步骤重置模型。这允许自适应控制，但不能实现持续的在线适应，因为之前步骤的更新总是被丢弃。在我们的比较中，我们发现我们的方法大大优于先前的方法。据我们所知，我们的工作第一个应用元学习来学习流式在线更新的工作。

3 问题陈述

我们将在线学习问题设置形式化如下：在每个时间步骤，模型接收输入 $\mathbf{x}_t$ 并生成预测 $\hat{\mathbf{y}}_t$ 。然后，它接收一个真实标签 $\mathbf{y}_t$ ，该标签必须用于调整模型以提高对下一个输入 $\mathbf{x}_{t+1}$ 的预测准确性。假设真实标签来自某个任务分布 $P(Y_t|X_t, T_t)$ ，其中 T_t 是时间 t 的任务。任务本身随着时间的推移而变化，导致任务分布不稳定，并且任务 T_t 的身份对于学习者来说是未知的。在现实世界中，任务可能对应于系统的未知参数（例如机器人的电机故障）、用户偏好或其他意外事件。该问题陈述涵盖了一系列在线学习问题，这些问题都需要不断适应流数据并在泛化和专业化之间进行权衡。

在我们的实验中，我们使用基于模型的强化学习作为工作示例，其中输入 $\mathbf{x}_t$ 是状态-动作对，输出 $\mathbf{y}_t$ 是下一个状态。我们在第 6 节中讨论了基于模型的 RL 的应用，但我们保留了针对任意在线预测问题的情况的通用方法的以下推导。

4 混合元训练网络的在线学习

我们分两部分讨论在线学习的元学习（MOLE）算法：本节中的在线学习和下一节中的元学习。在本节中，我们将解释我们的在线学习方法，该方法使用来自非平稳任务分布的连续输入数据流来实现有效的在线学习。我们的目标是保留泛化，以免丢失过去的知识，并获得专业化，这对于学习进一步不分布且需要更多学习的新任务尤为重要。我们在第二节讨论了获得元学习先验的过程。5，但我们首先在本节中使用 SGD 和期望最大化来制定在线适应算法，以维护和适应任务模型参数的混合模型（即概率任务分布）。

4.1 方法概述

让 $p_{\theta(T_t)}(\mathbf{y}_t|\mathbf{x}_t)$ 表示我们的模型在输入 $\mathbf{x}_t$ 上对于未知任务 T_t 的预测分布。我们的目标是估计非平稳任务分布中每个任务 T 的模型参数 $\theta_t(T)$ ：这需要推断每个步骤 $P(T_t|\mathbf{x}_t, \mathbf{y}_t)$ 的任务分布，使用该分布进行预测 $\hat{\mathbf{y}}_t = p_{\theta(T_t)}(\mathbf{y}_t|\mathbf{x}_t)$ ，并使用它来更新每个模型

$\theta_t(T)$ 到 $\theta_{t+1}(T)$ 。实际上，每个模型参数 $\theta(T)$ 将对应于神经网络 $f_{\theta(T)}$ 的权重。

每个模型都以一些先验参数向量 θ^* 开始，我们将在第 5 节中更详细地讨论它。由于任务数量也是未知的，因此我们从时间步 0 处的一个任务开始，其中 $\theta_0(T) = \{\theta_0(T_0)\} = \{\theta^*\}$ 。从这里开始，我们不断更新 $\theta_t(T)$ 中的所有参数并根据需要添加新任务，以尝试对真实的底层流程 $P(Y_t|X_t, T_t)$ 进行建模。由于任务身份未知，我们还必须在每个时间步估计 $P(T_t)$ 。因此，在线学习问题包括根据推断的任务概率 $P(T_t = T_i|\mathbf{x}_t, \mathbf{y}_t)$ 在每个时间步 t 调整每个 $\theta_t(T_i)$ 。为此，我们采用期望最大化 (EM) 算法并优化期望对数似然，由下式给出

$$\mathcal{L} = E_{T_t \sim P(T_t|\mathbf{x}_t, \mathbf{y}_t)} [\log p_{\theta_t(T_t)}(\mathbf{y}_t|\mathbf{x}_t, T_t)], \quad (1)$$

其中我们使用 $\theta_t(T_t)$ 来表示任务 T_t 对应的模型参数。最后，为了处理未知数量的任务，我们使用中餐馆流程根据需要实例化新任务。

4.2 近似在线推理

我们使用期望最大化 (EM) 来更新模型参数。在我们的例子中，EM 中的 E 步骤涉及估计当前时间步骤的任务分布 $P(T_t = T_i|\mathbf{x}_t, \mathbf{y}_t)$ ，而 M 步骤涉及更新 $\theta_t(T)$ 中的所有模型参数以获得新的模型参数 $\theta_{t+1}(T)$ 。根据推断的任务职责，参数始终在每个时间步更新一个梯度步。

我们首先估计任务分布中所有任务参数 T_i 的期望，其中每个任务概率的后验可以写成如下：

$$P(T_t = T_i|\mathbf{x}_t, \mathbf{y}_t) \propto p_{\theta_t(T_i)}(\mathbf{y}_t|\mathbf{x}_t, T_t = T_i)P(T_t = T_i). \quad (2)$$

然后，我们使用中餐馆流程 (CRP) 制定任务先验 $P(T_t = T_i)$ ，以便在试验期间实例化新任务。CRP 是狄利克雷过程的实例。在 CRP 中，在时间 t ，每个任务 T_i 的概率应由下式给出

$$P(T_t = T_i) = \frac{n_{T_i}}{t - 1 + \alpha} \quad (3)$$

其中 n_{T_i} 是任务 T_i 中所有步骤 $1, \dots, t-1$ 的预期数据点数量， α 是控制新任务实例化的超参数。因此先验变为

$$P(T_t = T_i) = \frac{\sum_{t'=1}^{t-1} P(T_{t'} = T_i)}{t - 1 + \alpha} \quad \text{and} \quad P(T_t = T_{\text{new}}) = \frac{\alpha}{t - 1 + \alpha} \quad (4)$$

结合先验和似然，我们得出以下后验任务概率分布：

$$P(T_t = T_i|\mathbf{x}_t, \mathbf{y}_t) \propto p_{\theta_t(T_i)}(\mathbf{y}_t|\mathbf{x}_t, T_t = T_i) \left[\sum_{t'=1}^{t-1} P(T_{t'} = T_i) + \delta(T_{t'} = T_{\text{new}})\alpha \right] \quad (5)$$

估计出潜在任务概率后，我们接下来执行 M 步骤，这会根据推断的任务分布改进等式 1 中的预期对数似然。由于每个任务都从先前的 θ^* 开始，因此一次梯度更新后 $\theta_t(T)$ 中所有参数的值由下式给出

$$\theta_{t+1}(T_i) = \theta^* - \beta \sum_{t'=0}^t P_t(T_{t'} = T_i|\mathbf{x}_{t'}, \mathbf{y}_{t'}) \nabla_{\theta_{t'}(T_i)} \log p_{\theta_{t'}(T_i)}(\mathbf{y}_{t'}|\mathbf{x}_{t'}) \quad \forall T_i \quad (6)$$

如果我们假设 $\theta_t(T)$ 的所有参数已经在之前的时间步 $0, \dots, t$ 中更新，我们可以通过简单地更新最新数据上的所有参数 $\theta_t(T)$ 来近似此更新：

$$\theta_{t+1}(T_i) = \theta_t(T_i) - \beta P_t(T_t = T_i|\mathbf{x}_t, \mathbf{y}_t) \nabla_{\theta_t(T_i)} \log p_{\theta_t(T_i)}(\mathbf{y}_t|\mathbf{x}_t) \quad \forall T_i \quad (7)$$

这个过程是一个近似过程，因为任务参数 $\theta_t(T)$ 的更新实际上也会改变之前时间步骤的相应任务概率。然而，这个近似

无需存储以前看到的数据点，并产生完全在线的流式算法。最后，为了完全实现 EM 算法，我们必须在每个时间步交替使用 E 和 M 步骤进行收敛，在每次迭代时将之前的梯度更新回滚到 $\theta_t(T)$ 。在实践中，我们发现每个时间步只执行一次 E 和 M 步骤就足够了。虽然这是一个粗略的简化，但在线学习场景中的连续时间步骤可能是相关的，使得这个过程合理。然而，在仍然保持完全在线的情况下执行多个 EM 步骤也很简单。

现在，我们总结了 MOLe 的完整在线学习部分，并在 Alg 中对其进行了概述。1. 在第一个时间步 $t = 0$ ，任务分配被初始化为包含一个条目： $\theta_0(T) = \{\theta_0(T_0)\} = \{\theta^*\}$ 。此后的每个时间步，执行 E 步骤来估计任务分布，并执行 M 步骤来更新模型参数。CRP 先验还在每个时间步分配在给定时间步添加新任务 T_{new} 的概率。这个新任务的参数 $\theta_{t+1}(T_{\text{new}})$ 是根据最新数据改编自 θ^* 。然后使用与最可能的任务 T^* 相对应的模型参数 $\theta_{t+1}(T^*)$ 对下一个数据点进行预测。

Algorithm 1 Online Learning with Mixture of Meta-Trained Networks

Require: θ^* from meta-training
Initialize $t = 0, \theta_0(T) = \{\theta_0(T_0)\} = \{\theta^*\}$
for each time step t **do**
 Calculate $p_{\theta_t(T_i)}(\mathbf{y}_t|\mathbf{x}_t, T_t = T_i) \forall T_i$
 Calculate $P_t(T_i) = P_t(T_t = T_i|\mathbf{x}_t, \mathbf{y}_t) \forall T_i$
 Calculate $\theta_{t+1}(T_i)$ by adapting from $\theta_t(T_i) \forall T_i$
 Calculate $\theta_{t+1}(T_{\text{new}})$ by adapting from θ^*
 if $P_t(T_{\text{new}}) > P_t(T_i) \forall T_i$ **then**
 Add $\theta_{t+1}(T_{\text{new}})$ to $\theta_{t+1}(T)$
 Recalculate $P_t(T_i)$ using $\theta_{t+1}(T_i) \forall T_i$
 Recalculate $\theta_{t+1}(T_i)$ using updated $P_t(T_i) \forall T_i$
 end if
 $T^* = \operatorname{argmax}_{T_i} p_{\theta_{t+1}(T_i)}(\mathbf{y}_t|\mathbf{x}_t, T_t = T_i)$
 Receive next data point $\mathbf{x}_{t+1}$
end for

5 元学习先验

我们制定了上面的算法，用于使用不断传入的数据执行在线适应。对于这种方法，我们选择使用与模型无关的元学习（MAML）算法对先验进行元训练。这种元训练算法是一个合适的选择，因为它产生了专门用于基于梯度的微调的先验。在我们进一步讨论元训练过程的选择之前，我们首先概述 MAML 和元学习。

给定任务分布，元学习算法会产生一个学习过程，在某些情况下，该学习过程可以快速适应新任务。MAML 针对深度网络的初始化进行了优化，当使用该任务中的一些数据点进行微调时，该网络可以实现良好的几次任务泛化。在训练时，MAML 会看到来自大量任务的少量数据，其中来自每个任务 T 的数据 $\mathcal{D}_T$ 可以分为训练和验证子集（ $\mathcal{D}_T^{\text{tr}}$ 和 $\mathcal{D}_T^{\text{val}}$ ），其中 $\mathcal{D}_T^{\text{tr}}$ 的大小为 k 。MAML 针对模型参数 θ 进行优化，以便 $\mathcal{D}_T^{\text{tr}}$ 上的一个或多个梯度步进会导致 $\mathcal{D}_T^{\text{val}}$ 上的损失最小化 L 。在我们的例子中，我们将设置 $\mathcal{D}_{T_t}^{\text{tr}} = (\mathbf{x}_t, \mathbf{y}_t)$ 和 $\mathcal{D}_{T_t}^{\text{val}} = (\mathbf{x}_{t+1}, \mathbf{y}_{t+1})$ ，损失 L 将对应于负对数似然。因此，一个好的 θ 允许这种适应在各种元训练任务中取得成功，这是一个良好的网络初始化，适应可以解决与先前看到的任务相关的各种新任务。MAML 目标定义如下：

$$\min_{\theta} \sum_T L(\theta - \eta \nabla_{\theta} L(\theta, \mathcal{D}_T^{\text{tr}}), \mathcal{D}_T^{\text{val}}) = \min_{\theta} \sum_T L(\phi_T, \mathcal{D}_T^{\text{val}}). \quad (8)$$

这里， η 是内部学习率。一旦这个元目标被优化，得到的 θ^* 就会充当先验，可以在测试时使用 $\mathcal{D}_{T_{\text{test}}}^{\text{tr}}$ 的最新经验进行微调，如下所示：

$$\phi_{T_{\text{test}}} = \theta^* - \eta \nabla_{\theta^*} L(\theta^*, \mathcal{D}_{T_{\text{test}}}^{\text{tr}}). \quad (9)$$

这里， $\phi_{T_{\text{test}}}$ 是根据元学习的先验 θ^* 改编的，以便更能代表当前时间。

尽管芬恩等人。(2017) 证明了深度神经网络的这种快速适应，Nagabandi 等人。(2018) 将此框架扩展到基于模型的元强化学习，这些方法解决了 k -shot 设置中的适应问题，始终直接从元学习先验中进行适应，并且不允许进一步适应或专门化。在这项工作中，我们通过暂时扩展的在线适应过程实现更多的知识进化，从而扩展了这些功能。

虽然我们的持续在线学习程序仍然是通过 k -镜头适应（即 MAML）的元训练来初始化的，但我们发现这个先验足以实现有效的持续学习

测试时在线适应。其直观的理由是，MAML 训练模型能够仅使用少量数据点和梯度步骤进行显著变化。请注意，这个元训练先验可以在 (a) k -shot 设置中的测试时使用，类似于它的训练方式，或者可以通过 (b) 采取远离该先验的更多梯度步骤来在测试时使用它。我们在第二节展示。从图 7 中可以看出，我们的方法优于这两种方法，但仅仅以这些方式使用先前元学习的能力就使得 MAML 的使用变得诱人。

我们注意到，很可能修改 MAML 算法以直接针对 4.2 节中讨论的加权更新优化模型。这仅需要在元训练期间计算每个批次的任务权重（E 步骤），然后构建一个计算图，其中所有梯度更新都乘以各自的权重。然后标准自动微分软件可以计算相应的元梯度。对于较短的试验长度，这并不比标准 MAML 复杂得多；对于较长的试验长度，可以选择截断反向传播。尽管这样的元训练过程更好地匹配在线适应期间使用模型的方式，但我们发现它并没有实质性地改善我们的结果。虽然如果进行长期适应的元训练，差异可能会更显著，但这一观察确实表明，简单地使用 MAML 进行元训练就足以在非平稳多任务设置中实现有效的连续在线适应。需要澄清的是，虽然这种合并 EM 权重更新的修改训练程序（在元训练期间）并没有显著改善我们的结果，但我们看到，通过为标准 MAML 元训练程序使用更多数据，测试时性能确实有所提高（参见附录）。

6 基于模型的强化学习的应用

在我们的实验中，我们将 MOLe 应用于基于模型的强化学习。一般来说，强化学习的目标是以最大化未来奖励总和的方式行事。在每个时间步 t ，代理从状态 $\mathbf{s}_t \in S$ 执行动作 $\mathbf{a}_t \in A$ ，根据转换概率（即动态） $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$ 转换到下一个状态 $\mathbf{s}_{t+1}$ 并接收奖励 $r_t = r(\mathbf{s}_t, \mathbf{a}_t)$ 。每一步的目标是执行操作 $\mathbf{a}_t$ ，以最大化未来奖励的折扣总和 $\sum_{t'=t}^{\infty} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'})$ ，其中折扣因子 $\gamma \in [0, 1]$ 优先考虑近期奖励。特别是在基于模型的强化学习中，来自已知或学习的动态模型的预测用于学习策略，或直接在规划算法中使用以选择最大化奖励的操作。

在我们的工作中，我们旨在建模的底层分布是动态分布 $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, T_t)$ ，其中未知的 T_t 代表底层设置（例如，系统状态、外部细节、环境扰动等）。MOLe 的目标是使用预测模型 p_θ 来估计该分布。为了在基于模型的强化学习环境中实例化 MOLe，我们遵循算法 1，并具有以下规范：

- (1) 我们将输入 $\mathbf{x}_t$ 设置为 K 先前状态和动作的串联，由 $\mathbf{x}_t = [\mathbf{s}_{t-K}, \mathbf{a}_{t-K}, \dots, \mathbf{s}_{t-2}, \mathbf{a}_{t-2}, \mathbf{s}_{t-1}, \mathbf{a}_{t-1}]$ 给出，输出为相应的下一个状态 $\mathbf{y}_t = [\mathbf{s}_{t-K+1}, \dots, \mathbf{s}_{t-1}, \mathbf{s}_t]$ 。与仅使用给定时间步长的数据相比，这为我们的每次在线更新提供了稍大的一批数据。由于高频下的各个时间步可能非常嘈杂，因此使用过去的 K 转换有助于抑制更新。
- (2) 预测模型 p_θ 将这些基础转换中的每一个表示为独立的高斯，使得 $p_\theta(\mathbf{y}_t|\mathbf{x}_t) = \prod_{t'=t-K}^{t-1} p(\mathbf{s}_{t'+1}|\mathbf{s}_{t'}, \mathbf{a}_{t'})$ ，其中每个 $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$ 都用由均值 $f_\theta(\mathbf{s}_t, \mathbf{a}_t)$ 和常数方差 σ^2 给出的高斯参数化。我们将这个平均动态函数 $f_\theta(\mathbf{s}_t, \mathbf{a}_t)$ 实现为一个神经网络模型，该模型具有三个隐藏层，每个隐藏层的维度为 500，并且具有 ReLU 非线性。
- (3) 为了计算新的任务参数 $\theta_{t+1}(T_{\text{new}})$ （可能会或可能不会添加到任务分布 $\theta_{t+1}(T)$ ），我们使用一组独立于集合 $\mathbf{x}_t$ 的附近数据点 K 。这样做是为了避免使用评估参数的同一数据集来计算参数，因为 $P_t(T_{\text{new}})$ 来自评估数据 $\mathbf{x}_t$ 上的参数。
- (4) 与仅给出下一个数据点 $\mathbf{x}_{t+1}$ 的标准在线流任务不同，这种情况下的传入数据点（即下一个访问的状态）受到预测模型本身的影响。这是因为，从可能的任务中选择最可能的任务 T^* 后，

控制器使用模型 $p_{\theta_{t+1}(T^*)}$ 来规划一系列未来动作 $\mathbf{a}_0, \dots, \mathbf{a}_H$ 并选择最大化未来奖励的动作。请注意，规划过程基于随机优化，遵循先前的工作（Nagabandi 等人，2018），我们在附录中提供了更多详细信息。由于控制器的动作选择决定了下一个数据点，并且由于控制器的选择取决于估计的模型参数，因此在这种情况下适当地调整模型就显得更加重要。

5) 最后，请注意，我们使用与模型无关的元学习（MAML）从元训练中获得了 θ^* ，如上面的方法中提到的。然而，在这种情况下，MAML 是在基于模型的 RL 循环中执行的。换句话说，控制器使用元训练给定迭代中的模型参数来生成策略上的部署，然后将这些部署中的数据添加到 MAML 的数据集中，并且重复此过程直到元训练结束。

7 实验

我们的实验旨在研究的问题包括：MOLe 1) 能否在非平稳数据流中自主发现某些任务结构？2) 适应比 k -shot 学习方法可以处理的任务分布更远的任务？3) 识别并恢复到以前见过的任务？4) 避免过度拟合最近的任务以防止下一次任务切换时性能下降？5) 优于其他方法？

为了研究这些问题，我们对 MuJoCo 物理引擎中的代理进行了实验（Todorov 等人，2012）。我们使用的代理是半猎豹（S R21、A R6）和六足爬行者（S R50、A R12）。使用这些代理，我们设计了许多具有挑战性的在线学习问题，这些问题涉及底层任务分布的多个突然和逐渐的变化，包括从以前看到的那些在线学习至关重要的任务中推断出来的任务。通过这些实验，我们的目标是构建能够代表真实强化学习智能体可能遇到的干扰和变化类型的问题设置。

我们在以下三个部分中介绍我们的研究结果和分析，视频可以在 <https://sites.google.com/berkeley.edu/onlineviameta> 找到。在我们的实验中，我们比较了几种替代方法，包括两种利用元训练的方法和两种不利用元训练的方法：

- (a) 通过元学习进行 k -shot 适应：始终根据元训练的先验 θ^* 进行适应，就像元学习方法通常所做的那样（Nagabandi 等人，2018）。这种方法通常不足以适应更加分散的任务，并且适应也不能及时进行以供将来使用。
- (b) 通过元学习继续适应：始终从前一个时间步骤的参数中采取梯度步骤。这种方法通常会过度拟合最近观察到的任务，因此它应该表明我们的方法有效识别任务结构以避免过度拟合并启用召回的重要性。
- (c) 基于模型的强化学习：使用标准监督学习，使用与上述方法相同的数据训练模型，并在整个试验过程中保持该模型固定（即没有元学习，也没有适应）。
- (d) 具有在线梯度更新的基于模型的强化学习：使用与基于模型的强化学习相同的模型（即无元学习），但在运行时使用梯度下降进行在线调整。这是常用动态评估方法的代表（Rei, 2015; Krause 等人，2017; 2016; Fortunato 等人，2017）。

7.1 半猎豹的地形坡度

我们从半猎豹（图 1）智能体的任务开始，穿越不同坡度的地形。先前的模型是对来自具有低幅度随机坡度的地形的数据进行元训练的，并且测试试验是在困难的分布外任务（例如盆地、陡峭的山丘等）上执行的。如图2所示，基于模型的强化学习和具有在线梯度更新的基于模型的强化学习在这些分布外任务上都表现不佳，即使这些任务

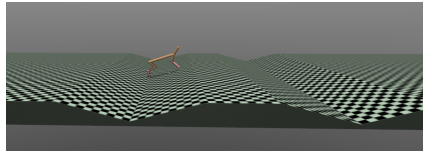


图 1: 半猎豹特工，显示正在穿越带有“盆地”的景观

模型使用元训练模型收到的相同数据进行训练。基于模型的 RL 方法的糟糕性能表明需要模型适应（而不是假设单个模型可以涵盖所有内容），而基于模型的 RL 和在线梯度更新的糟糕性能表明需要元学习初始化来实现神经网络的在线学习。

对于三种元学习和适应方法，我们预计元学习的持续适应会表现不佳，因为连续的梯度步骤会导致它过度拟合最近的数据；也就是说，我们预计上坡时的经验会导致下坡时的性能恶化，或类似的情况。然而，根据我们的定性和定量结果，我们发现元学习过程似乎使用参数空间初始化了代理，在该参数空间中，这些不同的“任务”并没有被视为有本质上的不同，其中 SGD 的在线学习表现良好。这表明元学习过程找到了一个任务空间，在该空间中，斜坡之间的技能可以轻松转移；因此，即使 MOLe 面临切换任务或向其动态潜在任务分布添加新任务的选择，它也选择不这样做（图 3）。与我们稍后将看到的发现不同，有趣的是，这里发现的任务空间并不对应于人类可区分的分类标签。最后，请注意，这些改变斜率的任务彼此并不是特别相似（并且发现的任务空间确实有用），因为尽管在浅训练斜率上具有相似的训练性能，但两个非元学习基线确实在这些测试任务上失败了。

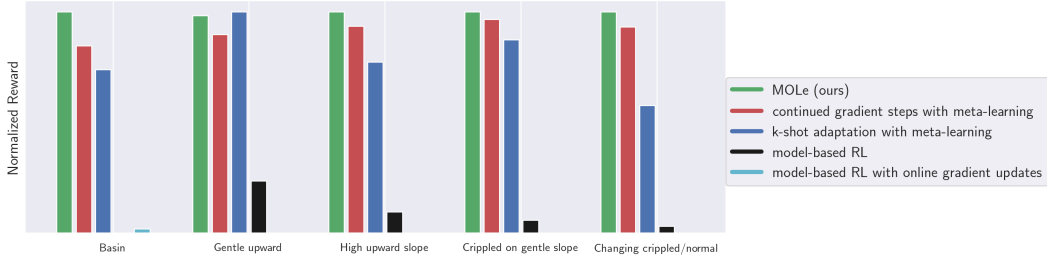


图 2：半猎豹地形穿越结果。性能不佳的基于模型的强化学习表明单个模型是不够的，而具有在线梯度更新的基于模型的强化学习表明元学习初始化至关重要。三种元学习方法的表现相似；然而，当任务需要远离先验的多个梯度步骤时，k-shot 自适应的性能就会恶化。

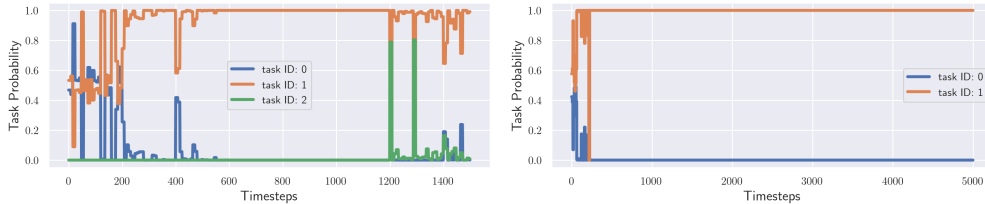


图 3：两个半猎豹景观遍历任务随时间变化的潜在任务分布，其中每次运行中遇到的地形坡度都不同。有趣的是，我们发现 MOLe 选择仅使用单个潜在任务变量来描述变化的地形。

7.2 半猎豹电机故障

虽然半猎豹在倾斜地形上的发现表明，对于表面上看起来像是单独的任务来说，单独的任务参数并不总是必要的，但我们还想研究在现实世界中经历更剧烈变化的非平稳任务分布的智能体。对于这组实验，我们根据数据训练所有模型，其中随机选择单个执行器以在推出过程中遇到故障。在这种情况下，故障意味着施加到该致动器的动作的极性或幅度被改变。图 4 显示了各种方法在完全不符合分布的测试任务上的结果，例如立即更改所有执行器。图 4 左侧显示，当测试期间的任务分布仅包含单个任务时，例如“负号”，其中所有执行器都被规定为相反极性，则通过对传入数据重复执行梯度更新，持续适应效果良好。然而，如图 4 的其他任务所示，这种持续适应的性能

当智能体经历非平稳的任务分配时，性能会大幅恶化。由于对最近传入数据的过度专业化，这种不断适应的方法往往会忘记并失去以前存在的技能。图 4 所示的持续性能恶化也说明了这种对过去技能的过度拟合和遗忘。另一方面，MOLe 动态构建概率任务分布，并允许适应这些困难的任务，而不会忘记过去的技能。我们在图 5 中展示了一个示例任务设置，其中代理经历了正常腿操作和残腿操作的交替时期。该图显示了新任务和旧任务的成功识别；请注意，识别和适应都是在线完成的，没有使用过去的数据库来执行适应，也没有人类指定的一组任务类别。

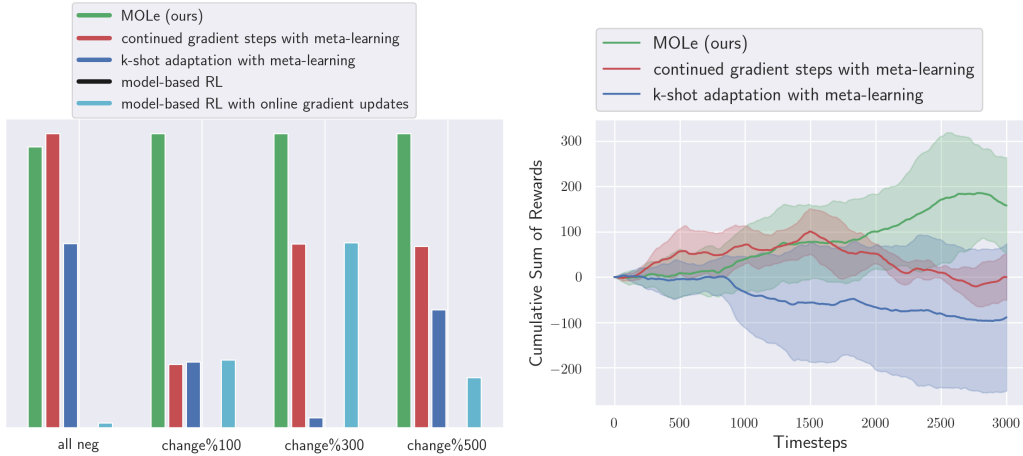


图 4: 电机故障试验的结果，其中不同的试验显示了以不同频率调制的任务分布（或对于第一类保持恒定）。在这里，在线学习对于良好的性能至关重要，k-shot 适应对于此类不同的任务来说是不够的，并且持续的梯度步骤会导致对最近看到的数据的过度拟合。

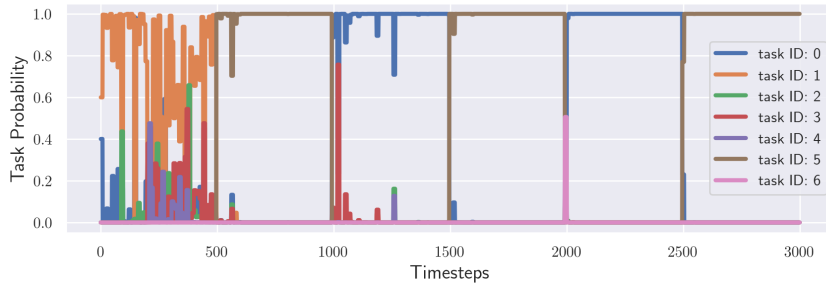


图 5: 在线学习试验过程中的潜在任务变量分布，其中潜在的运动故障每 500 个时间步长发生变化。我们发现 MOLe 能够成功恢复任务结构，识别底层任务何时发生变化，并回忆以前见过的任务。

7.3 六足履带机末端执行器的损坏

为了进一步检查我们的持续在线适应算法的效果，我们研究了另一种更复杂的代理：六足爬虫（图 6）。在这些实验中，模型是在随机受损关节（即无法应用执行器命令）上进行训练的。在图 7 中，我们提出了两个说明性测试任务：（1）代理在其测试时间体验期间看到一组残废配置，（2）代理在正常操作区域和残腿区域之间接收交替的体验周期。

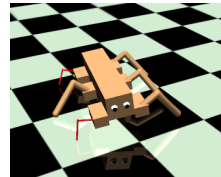


图 6: 六足履带式机器人，显示腿部有残缺。

第一个设置与训练期间看到的数据类似，因此，我们看到即使是基于模型的强化学习和具有在线梯度更新基线的基于模型的强化学习也不会失败。包括元学习和自适应的方法

然而，它确实具有更高的性能。此外，我们再次看到，在单任务设置的情况下，连续的梯度步骤并不是有害的。第二个设置的非平稳任务

分布（当腿部瘫痪是动态的）说明了在线适应的需要（基于模型的强化学习失败）、需要良好的先验适应（基于模型的强化学习在线梯度更新失败）、过度拟合最近的经验并因此忘记旧技能的危害（连续梯度步骤的低性能），以及远离先验的进一步适应的需要（k-shot 适应的性能有限）。借助 MOLe，该代理能够构建自己的“任务”开关表示，并且我们看到该开关确实对应于识别腿部残废区域（图 7.3 左侧）。三种元学习加适应方法的奖励累积和图（图 7.3 右侧）每 500 步都包含相同的任务切换模式：在这里，我们可以清楚地看到步骤 500-1000 和 1500-2000 是残缺区域。连续梯度步骤实际上在第二次和第三次正常操作时表现更差，而 MOLe 明显更好，因为它更频繁地看到任务。请注意这两种技能的提高，其中一种技能的发展实际上并不妨碍另一种技能。

最后，我们检查了爬行器（在每次试验期间）直行、转弯、有时腿瘸的实验。对于连续梯度步骤，“以正常配置向前行走”的前 500 个时间步长的性能与 MOLe 相当（ $\pm 10\%$ 差异），但“以正常配置向前行走”的最后 500 个时间步长的性能要低 200%。请注意，在不允许单独的任务专业化/适应的情况下执行更新会产生不利影响。

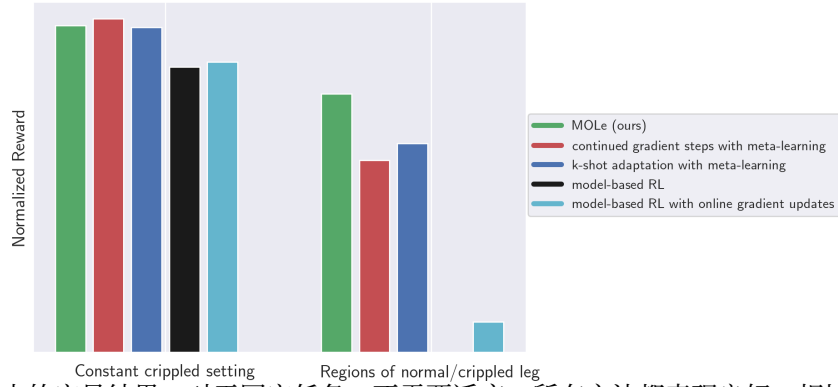


图 7：爬虫的定量结果。对于固定任务，不需要适应，所有方法都表现良好。相比之下，当任务在试验中动态变化时，只有 MOLe 可以有效地在线学习。

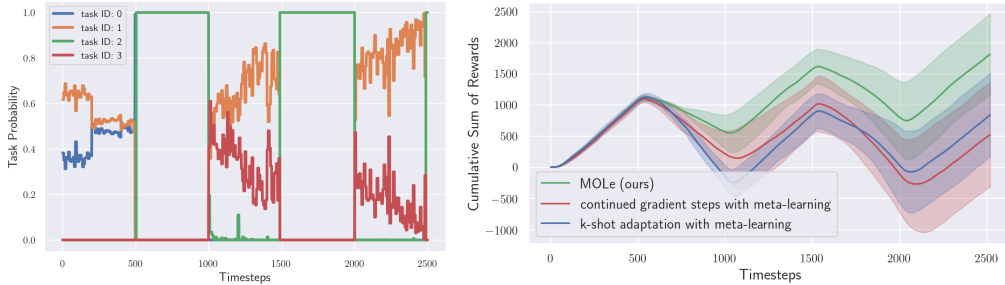


图 8：爬行器实验结果。左：在线识别正常/残障经历交替时期的潜在任务概率。右图：MOLe 通过多次执行相同的任务而得到改进。

8 讨论

我们提出了一种神经网络模型的在线学习方法，可以处理每次试验中的非平稳、多任务设置。我们的方法直接使用 SGD 来调整模型，其中 EM 算法在维持任务分布并处理非平稳性之前使用中餐馆流程。尽管 SGD 通常在大型参数模型（例如深度神经网络）的流设置中导致在线学习算法较差，但我们观察到，通过（1）对模型进行元训练以使用 MAML 快速适应，以及（2）在测试时使用我们的在线算法进行概率更新，我们可以使用神经网络实现有效的在线学习。在我们的实验中，我们将这种方法应用于基于模型的强化学习，并证明它可以用于适应模拟机器人面临各种新的和意外任务的行为。我们的

结果表明, 我们的方法可以发展自己的任务概念, 根据需要不断适应先前的任务 (甚至学习需要更多适应的任务), 并回忆起以前见过的任务。虽然我们使用基于模型的强化学习作为我们的评估领域, 但我们的方法是通用的, 可以应用于其他流媒体和在线学习设置。未来工作的一个令人兴奋的方向是将我们的方法应用于时间序列建模和主动在线学习等领域。

参考

- Maruan Al-Shedivat, Trapit Bansal, Yuri Burda, Ilya Sutskever, Igor Mordatch 和 Pieter Abbeel。通过元学习在非平稳和竞争环境中持续适应。 *arXiv preprint arXiv:1710.03641*, 2017 年。
- Rahaf Aljundi, Punarjay Chakravarty 和 Tinne Tuytelaars。专家门: 通过专家网络进行终身学习。在 *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017 年。
- 萨米·本吉奥、约书亚·本吉奥、乔斯林·克劳蒂尔和扬·格塞伊。关于突触学习规则的优化。在 *Optimality in Artificial and Biological Neural Networks*, 1992 年。
- 大卫·M·布莱 (David M Blei)、安德鲁·Y·吴 (Andrew Y Ng) 和迈克尔·乔丹 (Michael I Jordan)。潜在狄利克雷分配。 *Journal of machine Learning research*, 3 (一月): 993–1022, 2003。
- Zdravko I Botev, Dirk P Kroese, Reuven Y Rubinstein 和 Pierre LEcuyer。用于优化的交叉熵方法。在 *Handbook of statistics*, 第 31 卷, 第 35–59 页。爱思唯尔, 2013。
- 莱恩·博图。在线学习和随机近似。 *On-line learning in neural networks*, 17(9):142, 1998。
- 塔玛拉·布罗德里克、尼古拉斯·博伊德、安德烈·维比索诺、阿希亚·C·威尔逊和迈克尔·乔丹。流变分贝叶斯。 *Advances in Neural Information Processing Systems*, 第 1727–1735 页, 2013 年。
- 朱利安·多扬和哈比卜·贝纳利。学习运动技能期间成人脑的重组和可塑性。 *Current opinion in neurobiology*, 15(2): 161–167, 2005。
- 严端、约翰·舒尔曼、陈曦、Peter L Bartlett、Ilya Sutskever 和 Pieter Abbeel。RL2: 通过慢速强化学习实现快速强化学习。 *arXiv:1611.02779*, 2016。
- 约翰·杜奇、埃拉德·哈赞和约拉姆·辛格。用于在线学习和随机优化的自适应次梯度方法。 *Journal of Machine Learning Research*, 2011。
- 切尔西·芬恩、彼得·阿贝尔和谢尔盖·莱文。与模型无关的元学习, 用于快速适应深度网络。 *International Conference on Machine Learning (ICML)*, 2017 年。
- J·兰德尔·弗拉纳根 (J Randall Flanagan) 和艾伦·M·温 (Alan M Wing)。在点对点手臂运动期间用负载力调节握力。 *Experimental Brain Research*, 95(1): 131–143, 1993。
- 梅雷·福图纳托、查尔斯·布伦德尔和奥利奥尔·维尼亚尔斯。贝叶斯循环神经网络。 *arXiv preprint arXiv:1704.02798*, 2017 年。
- 罗伯特·赫尔曼。 *Neural control of locomotion*, 第 18 卷。施普林格, 2017 年。
- 马修·霍夫曼、弗朗西斯·R·巴赫和大卫·M·布莱。潜在狄利克雷分配的在线学习。 *advances in neural information processing systems*, 第 856–864 页, 2010 年。
- 阿米尔·贾法里、艾米·格林沃尔德、大卫·冈德克和居内斯·埃卡尔。关于无悔学习、虚构游戏和纳什均衡。 *ICML*, 第 1 卷, 第 226–233 页, 2001 年。
- 加森·杰菲尔、艾琳·格兰特、托马斯·L·格里菲斯和凯瑟琳·海勒。用于元学习中传递调制的在线基于梯度的混合。 *arXiv preprint arXiv:1812.06080*, 2018。
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska 等。克服神经网络中的灾难性遗忘。 *Proceedings of the National Academy of Sciences*, 2017。

本·克劳斯、梁路、伊恩·默里和史蒂夫·雷纳斯。用于序列建模的乘法 lstm。
arXiv preprint arXiv:1609.07959, 2016 年。

本·克劳斯、伊曼纽尔·卡亨布韦、伊恩·默里和史蒂夫·雷纳斯。神经序列模型的动态评估。
CoRR, abs/1709.07432, 2017。

大卫·洛佩兹·帕兹等人。用于持续学习的梯度情景记忆。在 *Advances in Neural Information Processing Systems*, 2017 年。

Tsendsuren Munkhdalai 和洪宇。元网络。 *International Conference on Machine Learning (ICML)*, 2017 年。

K·墨菲。 *Dynamic Bayesian Networks: Representation, Inference and Learning*。 博士论文, 计算机科学系, 加州大学伯克利分校, 2002 年。 URL <https://www.cs.ubc.ca/murphyk/Thesis/thesis.html>。

阿努沙·纳加班迪、伊格纳西·克拉维拉、Simin Liu、罗纳德·S·费林、彼得·阿贝尔、谢尔盖·莱文和切尔西·芬恩。通过元强化学习学习适应动态的现实环境。
arXiv preprint arXiv:1803.11347, 2018。

Devang K Naik 和 RJ Mammone。通过学习来学习的元神经网络。在 *International Joint Conference on Neural Networks (IJCNN)*, 1992 年。

Cuong V Nguyen、Yingzhen Li、Thang D Bui 和 Richard E Turner。变分持续学习。
arXiv:1710.10628, 2017 年。

A.拉奥。最优控制数值方法综述。在 *Advances in the Astronautical Sciences*, 2009 年。

萨钦·拉维和雨果·拉罗谢尔。优化作为小样本学习的模型。在 *International Conference on Learning Representations (ICLR)*, 2017 年。

西尔维斯特·阿尔维斯·雷布菲、亚历山大·科列斯尼科夫和克里斯托夫·H·兰伯特。 icarl: 增量分类器和表示学习。在 *Proc. CVPR*, 2017 年。

马雷克·雷伊。循环神经语言模型中的在线表示学习。 *CoRR*, abs/1508.03854, 2015。

Samuel Ritter、Jane X Wang、Zeb Kurth-Nelson、Siddhant M Jayakumar、Charles Blundell、Razvan Pascanu 和 Matthew Botvinick。去过那里, 做过这样的事: 通过情景回忆进行元学习。
arXiv preprint arXiv:1805.09692, 2018。

安德烈·A·鲁苏、尼尔·C·拉比诺维茨、纪尧姆·德贾丁斯、休伯特·索耶、詹姆斯·柯克帕特里克、科雷·卡武克库奥格鲁、拉兹万·帕斯卡努和拉亚·哈塞尔。渐进神经网络。
arXiv:1606.04671, 2016 年。

Doyen Sahoo、Quang Pham、Jing Lu 和 Steven CH Hoi。在线深度学习: 动态学习深度神经网络。
arXiv preprint arXiv:1711.03705, 2017 年。

亚当·桑托罗、谢尔盖·巴图诺夫、马修·博特维尼克、达安·维尔斯特拉和蒂莫西·利利克拉普。使用记忆增强神经网络进行元学习。在 *International Conference on Machine Learning (ICML)*, 2016 年。

于尔根·施米德胡贝尔。自我参照学习的进化原则。 *Diploma thesis, Institut f. Informatik, Tech. Univ. Munich*, 1987。

弗洛里安·斯汀伯格、安德烈亚斯·鲁托和曼弗雷德·奥珀。具有可重用状态的动态系统中变化点的贝叶斯推理——一种中餐馆流程方法。 *Artificial Intelligence and Statistics*, 第 1117–1124 页, 2012 年。

塞巴斯蒂安·特龙。终身学习算法。在 *Learning to learn* 中。施普林格, 1998。

塞巴斯蒂安·特龙和洛里恩·普拉特。 *Learning to learn*。施普林格科学与商业媒体, 1998。

伊曼纽尔·托多罗夫、汤姆·埃雷兹和尤瓦尔·塔萨。Mujoco: 用于基于模型的控制的物理引擎。 *Intelligent Robots and Systems (IROS), 2012 IEEE/RSJ International Conference on*, 第 5026–5033 页。IEEE, 2012。Jane X Wang、Zeb Kurth-Nelson、Dhruva Tirumala、Hubert Soyer、Joel Z Leibo、Remi Munos、Charles Blundell、Dharshan Kumaran 和 Matt Botvinick。学习强化学习。 *arXiv:1611.05763*, 2016。王玉雄、Deva Ramanan 和 Martial Hebert。培养大脑：通过增加模型容量进行微调。在 *IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017 年。

格雷迪·威廉姆斯、安德鲁·奥尔德里奇和伊万杰洛斯·西奥多鲁。使用协方差变量重要性采样进行模型预测路径积分控制。 *CoRR*, abs/1509.01149, 2015。

弗里德曼·赞克、本·普尔和苏里亚·甘古利。通过突触智能持续学习。在 *International Conference on Machine Learning*, 2017 年。

测试时表现与训练数据

我们在下面验证了，随着元训练模型使用更多数据进行训练，它们在测试任务上的性能确实有所提高。

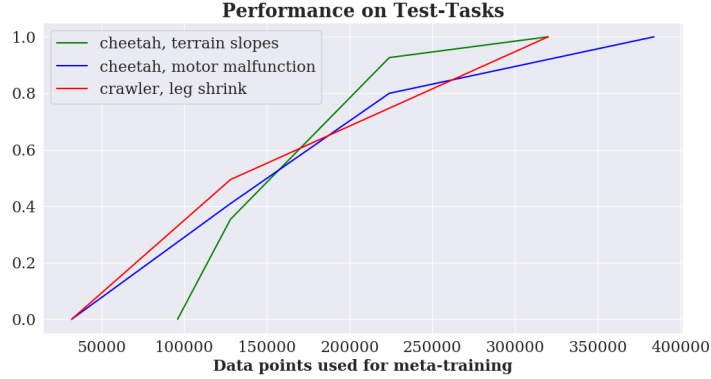


图 9: 使用不同数据量进行元训练的模型在测试任务（即训练期间看不见的）上的性能。这里的性能数字按每个代理进行标准化，介于 0 和 1 之间。

B 超参数

在所有实验中，我们使用由三个隐藏层组成的动力学模型，每个隐藏层的维度为 500，具有 ReLU 非线性。我们使用的控制方法是随机射击模型预测控制（MPC），其中在每个时间步对每个水平长度 $H=10$ 的 1000 个候选动作序列进行采样，通过预测模型进行反馈，并按其预期奖励进行排名。然后执行得分最高的候选操作序列中的第一个操作步骤，然后在下一个时间步骤再次重复整个规划过程。

下面，我们列出了实验中使用的各种方法的相关训练和测试参数。# Task/itr 对应于每次迭代收集数据以训练模型时采样的任务数量，# TS/itr 是该迭代期间收集的总步数（所有任务的总和）。

表 1: 训练时间的超参数

	Iters	Epochs	# Tasks/itr	# TS/itr	K	outer LR	inner LR (η)
Meta-learned approaches (3)	12	50	16	2000-3000	16	0.001	0.01
Non-meta-learned approaches (2)	12	50	16	2000-3000	16	0.001	N/A

表 2: 运行时的超参数

	α (CRP)	LR (model update)	K (previous data)
MOLe (ours)	1	0.01	16
continued adaptation with meta-learning	N/A	0.01	16
k-shot adaptation with meta-learning	N/A	0.01	16
model-based RL	N/A	N/A	N/A
model-based RL with online gradient updates	N/A	0.01	16

控制器

正如第 6 节中提到的，我们将学习到的动力学模型与控制器结合使用来选择下一个要执行的操作。控制器使用学习模型 f_θ 以及编码所需任务的奖励函数 $r(\mathbf{s}_t, \mathbf{a}_t)$ 。许多方法可以用来执行这种动作选择，包括交叉熵方法（CEM）（Botev等，2013）或模型预测路径积分控制（MPPI）（Williams等，2015），但在我们的实验中，我们使用随机采样射击方法 Rao（2009）。

在每个时间步 t ，我们随机生成 N 候选动作序列，每个序列中包含 H 个动作。

$$A_t^i = \{\mathbf{a}_t^i, \dots, \mathbf{a}_{t+H}^i\} \quad (10)$$

然后，我们使用学习到的动态模型来预测执行这些候选动作序列的结果状态。

$$S^i = \{\hat{\mathbf{s}}_{t+1}^i, \dots, \hat{\mathbf{s}}_{t+H+1}^i\} \text{ where } \hat{\mathbf{s}}_{p+1}^i = f_\theta(\hat{\mathbf{s}}_p^i, \mathbf{a}_p^i) \quad (11)$$

接下来，我们使用奖励函数来选择具有最高相关预测奖励的动作序列。

$$i^* = \operatorname{argmax}_i \sum_{t'=t}^{t'+H} r(\hat{\mathbf{s}}_{t'}^i, \mathbf{a}_{t'}^i) \quad (12)$$

接下来，我们使用模型预测控制 (MPC) 框架来仅执行当前状态 $\mathbf{s}_t$ 中的第一个动作 $\mathbf{a}_t^{i^*}$ ，而不是执行所选最佳动作的整个序列 $A_t^{i^*}$ 。然后我们在下一个时间步重新计划；MPC 的使用可以通过防止累积错误来补偿模型的不准确性，因为我们使用更新的状态信息在每个时间步重新规划。